Tab 1

CSE221
Previous Quiz Questions

Lab Topics

1. Sorting (Total Questions: 13)
2. Two Pointer, Searching (Total Questions: 19)
3. Divide and Conquer (Total Questions: 15)
4. Graph Traversal (BFS, DFS) and their Applications (Total Questions: 11)
5. Dijkstra (Total Questions: 11)
6. DSU, Minimum Spanning Tree
7. Greedy, Dynamic Programming

Note: We will gradually update this document with Quiz questions on the upcoming topics from the previous semester(s).

Previous Quiz Questions on Sorting

#1

Problem Proposed/Prepared by: ARD, KKP

You are given a file "**inp2.txt**". The file contains the following info: In the first line there is an integer **N (0 < N < 1000)**. In the second line, there are **N** distinct integers (the absolute value of each integer is less than 1000) separated by spaces.

You need to find the minimum number of swapping operations needed to sort these **N** integers in ascending order.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

There should be only one integer (the answer you were asked to find above) in the first and only line.

Sample Input File 1	Sample Output File 1
10 17 42 89 23 67 -15 78 910 34 50	8

Sample Input File 2	Sample Output File 2
7 -2 8 32 128 4 16 64	4

#2

Problem Proposed/Prepared by: ARD, KKP

You are given a file "**input2.txt**". The file contains the following info: In the first line there are two integers **N** and **K (0 < K < N < 1000)**. In the second line, there are **N** distinct integers (the absolute value of each integer is less than 1000) separated by spaces.

You need to find if it is possible to sort these **N** integers in descending order using no more than **K** swapping operations.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.txt**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one string "**Possible**" (if the above mentioned thing is possible) or "**Impossible**" (otherwise) in the first and only line.

Sample Input File 1	Sample Output File 1
10 7 17 42 89 23 67 -15 78 910 34 50	Impossible
Sample Input File 2	Sample Output File 2
7 6 -2 8 32 128 4 16 64	Possible

#3

Problem Proposed/Prepared by: ARD, KKP

You are given a file "**inp2.txt**". The file contains the following info: In the first line there is an integer **N (0 < N < 1000)**. In the second line there are **N** integers (absolute value of each integer is less than 1000) separated by spaces.

Let **Arr** be a list with these **N** integers in the given order and **Brr** be another list that is initially empty. You can remove either the first integer or the last integer from **Arr** and append it to the end of list **Brr**, as long as there are some integers remaining in **Arr**. Using this operation repeatedly, you remove all the integers from **Arr** and fill up the list **Brr**. Find if it is possible to make a list sorted in ascending order at the end in **Brr** in this way.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**"

(**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one string "**Possible**" (if the above mentioned thing is possible) or "**Impossible**" (otherwise) in the first and only line.

Sample Input File 1	Sample Output File 1
10 17 42 89 23 67 -15 78 910 34 50	Impossible
Sample Input File 2	Sample Output File 2
8 -2 8 16 32 128 64 16 4	Possible

#4

Problem Proposed/Prepared by: ARD, KKP

You are given a file "**input2.txt**". The file contains the following info: In the first line there is an integer **N (0 < N < 1000)**. In the second line there are **N** integers (absolute value of each integer is less than 1000) separated by spaces.

Suppose, there is a list with these **N** integers in the given order. In one operation, you can swap two integers in this list if, either both integers are in odd positions, or both integers are in even positions. Find if it is possible to sort the integers in ascending order by using this operation as many times as necessary.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.txt**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one string "**Possible**" (if the above mentioned thing is possible) or "**Impossible**" (otherwise) in the first and only line.

Sample Input File 1	Sample Output File 1
10 17 42 89 23 67 -15 78 910 34 50	Impossible
Sample Input File 2	Sample Output File 2
8 16 4 4 8 -2 32 64 128	Possible

#5

Problem Proposed/Prepared by: ARD, KKP

You are given a file "**inp2.txt**". The file contains the following info: In the first line there is an integer **N (0 < N < 1000)**. In the second line there are **N** distinct integers (absolute value of each integer is less than 1000) separated by spaces.

Suppose all these **N** integers are coming at you, one at a time, in the given order. For each integer, you need to find the rank of the integer, in a list containing that integer along with all the integers which came before it. The rank of an element is defined as one more than the number of elements that are greater or equal to that element that came before it.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be **N** integers (the answers you were asked to find above) in the first and only line.

Sample Input File 1	Sample Output File 1
10 17 42 89 23 67 -15 78 910 34 50	1 1 1 3 2 6 2 1 6 5
Sample Input File 2	Sample Output File 2
7 -2 8 32 128 4 16 64	1 1 1 1 4 3 2

#6

Problem Proposed/Prepared by: ARD, KKP

You are given a file "**input2.txt**". The file contains the following info: In the first line there are two integers **N** and **K (0 < N < 1000, 0 < k < 5)**. In the second line there are **N** integers (absolute value of each integer is less than 1000) separated by spaces.

Suppose, there is a list with these **N** integers in the given order. In one operation, you can reverse any **K** or less adjacent integers (any subarray of size **K** or less) in the list. Find if it is possible to sort the integers in ascending order by using this operation as many times as necessary.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.txt**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one string "**Possible**" (if the above mentioned thing is possible) or "**Impossible**" (otherwise) in the first and only line.

Sample Input File 1	Sample Output File 1

10 1 17 42 89 23 67 -15 78 910 34 50	Impossible
Sample Input File 2	Sample Output File 2
7 4 -2 8 32 128 4 16 64	Possible

#7

Problem Proposed/Prepared by: MLH, NAN

You are given a file "**inp2.txt**". The file contains the following info: In the first line there are two integers **N, M (0 < N < 5000, 0 < N < M < 500000)**. In the second line there are **N** distinct integers, each between **0** and **(M-1)**.

There is a list containing these **N** integers. Find the integers between **0** and **(M-1)** which are missing in this list in descending order. Solve the problem in **O(N^2+M)** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In the first and only line of this file there should be the missing integers, separated by spaces and sorted in descending order.

Sample Input File 1	Sample Output File 1
3 5 4 2 1	3 0
Sample Input File 2	Sample Output File 2
4 5 2 4 3 1	0
Sample Input File 3	Sample Output File 3
11 16 9 6 4 2 12 3 5 7 1 15 13	14 11 10 8 0

#8

Problem Proposed/Prepared by: MTJ, MNW

You are given a file "**inp2.txt**". The file contains the following info: In the first line there are two integers **N, M (0 < N < 5000, 0 < M < 500000)**. In the second line there are **N** integers, each between **1** and **500000**.

There is a list containing these **N** integers. Arrange these integers such that after arranging, the ones which are strictly greater than **M** are at the beginning of the list sorted in descending order, and the remaining ones are at the end of the list in their initial order. Solve the problem in **O(N^2)** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In the first and only line of this file there should be the integers of the list in the arranged order, separated by spaces.

Sample Input File 1	Sample Output File 1
5 3 4 2 1 5 3	5 4 2 1 3
Sample Input File 2	Sample Output File 2
4 3 2 4 3 10	10 4 2 3
Sample Input File 3	Sample Output File 3
11 160000 9 6 400000 2 12 300000 5 7 1 15 13	400000 300000 9 6 2 12 5 7 1 15 13

#9

Problem Proposed/Prepared by: ANV, HFN

You are given a file "**inp2.txt**". The file contains the following info: In the first line there are two integers **N, M (0 < M < N < 5000)**. In the second line there are **N** integers, each between **1** and **500000**.

There is a list containing these **N** integers. Among these, you can remove **M** integers and get the sum of the remaining ones. Find the difference between the minimum sum and the maximum sum you can get using this method. Solve the problem in **O(N^2)** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one integer, the difference you are asked to find above.

Sample Input File 1	Sample Output File 1
5 3 4 2 1 5 3	6
Sample Input File 2	Sample Output File 2
4 3 2 4 3 10	8
Sample Input File 3	Sample Output File 3

11 6 9 6 400000 2 12 300000 5 7 1 15 13	700019

#10

Problem Proposed/Prepared by: KKP

You are given a file "**inp2.txt**". The file contains the following info: In the first line there is an integer **N (0 < N < 5000)**. In the second line there are **N** distinct integers, each between **1** and **500000**.

There is a list containing these **N** integers. Arrange these integers such that after arranging, except for the first integer and the last integer, each integer is either greater than both its next and previous integers, or less than both its next and previous integers. Solve the problem in **O(N^2)** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In the first and only line of this file there should be the integers of the list in the arranged order, separated by spaces.

Sample Input File 1	Sample Output File 1
5 4 2 1 5 3	2 3 1 5 4
Sample Input File 2	Sample Output File 2
4 2 4 3 10	10 3 4 2
Sample Input File 3	Sample Output File 3
11 9 6 400000 2 12 300000 5 7 1 15 13	400000 9 300000 6 12 2 7 5 15 1 13

#11

Problem Proposed/Prepared by: KKP, ARD

You are given a file "**inp2.txt**". The file contains the following info: In the first line there are two integers **N, M (0 < M < N < 5000)**. In the second line there are **M** distinct integers, each between **1** and **N**. In the third line there are **N** distinct integers, each between **1** and **N**.

There is a list containing these **N** integers in the third line. In one swap, you can choose two distinct index numbers and then swap the two integers in those two indexes in the list. Using **one or more** such swaps, you need to arrange these integers such that after arranging, the **M** integers in the second line are present as a subsequence (in the same order but not necessarily at consecutive positions) of the **N** integers.

Find the number of swaps and the pair of indexes (**0**-based) for each of the swaps. Solve the problem in **O(N^2)** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In the first line of this file there should be an integer, the number of swaps. In the second line there should be the first index of the pairs in the order of the swaps, separated by spaces, empty if there is no swap. In the third line there should be the second index of the pairs in the same order, separated by spaces, empty if there is no swap.

Sample Input File 1	Sample Output File 1
5 4 3 5 2 1 4 2 1 5 3	3 0 1 2 4 3 3
Sample Input File 2	Sample Output File 2
4 3 2 4 3 2 4 3 1	1 2 3
Sample Input File 3	Sample Output File 3
11 6 1 2 3 4 5 6 9 6 4 2 11 3 5 7 1 10 8	5 8 3 5 5 6 0 1 2 3 4

#12

Problem Proposed/Prepared by: SBDK

You are given a file "**inp2.txt**". The file contains the following info: In the first line there is an integer **N (4 < N < 5000)**. In the second line there are **N** distinct integers, each between **1** and **500000**.

There is a list containing these **N** integers. In one swap, you can choose two distinct index numbers and then swap the two integers in those two indexes in the list. However, you are not allowed to swap two adjacent integers. Using **one or more** such non-adjacent swaps, you need to sort the integers of the list in ascending order. Find the number of swaps and the pair of indexes (**0**-based) for each of the swaps. Solve the problem in **O(N^2)** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In the first line of this file there should be an integer, the number of swaps. In the second line there should be the first index of the pairs in the order of the swaps, separated by spaces, empty if there is no swap. In

the third line there should be the second index of the pairs in the same order, separated by spaces, empty if there is no swap.

Sample Input File 1	Sample Output File 1
5 4 2 1 5 3	3 0 0 0 3 4 2
Sample Input File 2	Sample Output File 2
6 2 3 4 10 11 12	2 4 4 2 2
Sample Input File 3	Sample Output File 3
11 9 6 400000 2 11 300000 5 7 1 10 8	9 0 1 2 4 5 10 10 6 7 8 3 6 7 10 7 8 10 9

#13

Problem Proposed/Prepared by: HFN

You have been given a task to sort(bubble/insertion/selection) the array. But there is a catch: not all the values in your array need to be sorted. The numerical values in your arrays need to be sorted and other values will be kept as it is and add it at the end. Please devise an algorithm,to solve the problem. You have to use open() for input and output and the sample paths you can use for both will start as : **"c/user/FILENAME.txt"**

Input Description
In the input file there are two lines. In the first line there is only one integer **N**, indicating the number of items in the array. In the second line there are **N** items separated by spaces.

Output Description
In the output file, there will be only one line containing the items of the array in the asked order, each item separated by spaces.

Constraints

- $1 \leq N \leq 1000$
- The items in the array are either positive integers less than 1000, or a single capital letter A-Z.

Sample Input	Sample Output
10 7 2 C 3 A X 1 19 11 Z	1 2 3 7 11 19 C A X Z

Previous Quiz Questions on Two Pointers, Searching

#1

Problem Proposed/Prepared by: ASD

Imagine a path that has been paved with shades of light and dark tiles. Each tile's color can be either a dark shade or a light shade. You need to find out the minimum number of light tiles to be replaced by dark tiles so that there exists *k* consecutive dark tiles in the path.

Input:

The first line of input consists of a single integer *n* denoting the number of tiles in the path.

Second line consists of space separated *n* integers. The integers will be either 1 or 0. 1 means light tiles. 0 means dark tiles.

Third line of input consists of a single integer *k* denoting the number consecutive dark tiles there has to be in the path

Output:

Output consists of a single integer denoting the minimum number of light tiles that need to be replaced with black tiles. If the path already contains *k* consecutive dark tiles then the answer is 0.

ple Input 1	**Sample Output 1**
)1	1
ple Input 2	**Sample Output 2**
)1	2
ple Input 3	**Sample Output 3**
10	0
ple Input 4	**Sample Output 4**
	1

#2

Problem Proposed/Prepared by: KKP

Alice and Bob are two friends. Alice has a sorted list in ascending order of length **N.** On the other hand, Bob has a sorted list in ascending order of length **M.**

Now, they will merge these two sorted lists and sort those again. Can you find the **Kth** smallest element of the merged sorted list?

Come up with a solution that runs in O(k).

Input

The first line contains an integer N (1 <= N <= 10^5), denoting the length of Alice's sorted list. In the next line, there will be N integers separated by space.

The third line contains another integer M (1 <= M <= 10^5), denoting the length of Bob's sorted list. In the next line, there will be M integers separated by space.

The last line contains an integer K (1 <= K <= N+M) the smallest number you have to find from the merged sorted list.

All the numbers given in the input will fit in a 32-bit signed integer.

It is guaranteed that the given lists will be in sorted order.

Output:

Find the Kth smallest element after merging the two sorted lists.

Sample Input/Output:

Sample Input 1	Sample Output 1
4 1 3 5 7 4 2 2 4 8 5	4
Sample Input 2	Sample Output 2
3 2 10 12 6 3 4 6 7 8 9 8	10
Sample Input 3	Sample Output 3
4 1 2 3 4 1 10 2	2
Sample Input 4	Sample Output 4
7 2 3 8 8 10 12 14 9 1 1 4 5 6 8 13 15 16 15	15

#3

Problem Proposed/Prepared by: KKP

Alice, Bob, and Trudy are three friends. They have a list sorted in ascending order of length **U, V,** and **W,** respectively.

Now, they want to make a sorted list of **U+V+W** lengths in ascending order.

Your marks will depend on the correctness and efficiency of your code.

Input

The first line contains an integer U (1 <= U <= 10^5), denoting the length of Alice's sorted list. In the next line, there will be U integers separated by space.

The third line contains another integer V (1 <= V <= 10^5), denoting the length of Bob's sorted list. In the next line, there will be V integers separated by space.

The fifth line contains another integer W (1 <= W <= 10^5), denoting the length of Trudy's sorted list. In the next line, there will be W integers separated by space.

All the numbers given in the input will fit in a 32-bit signed integer.

It is guaranteed that the given lists will be in sorted order.

Output:

You have to make a sorted list in ascending order from the given lists in ascending order and show the output.

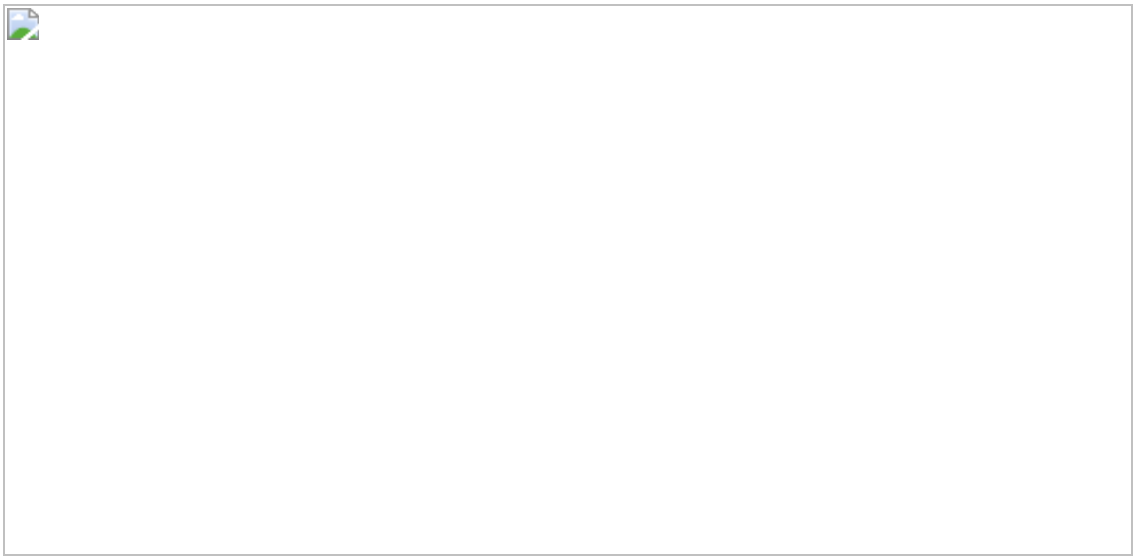
Sample Input/Output:

Sample Input 1	Sample Output 1
3 1 3 5 7 5 2 2 4 8 15 4 1 7 9 10	1 1 2 2 3 4 5 7 7 8 9 10 15
Sample Input 2	Sample Output 2
3 2 10 12 6 3 4 6 7 8 9 2 1 5	1 2 3 4 5 6 7 8 9 10 12

#6

Problem Proposed/Prepared by: ATY

You're a contestant in a game show set in a magical forest. In this forest, there are mystical water fountains scattered around, but they're guarded by mischievous creatures. To collect water for your village, you need to place two magic orbs between two trees. Each orb's height is limited by the shorter tree, and the width is the distance between the trees. Your goal is to maximize water collection to save your village from drought. Can you figure out the best placement for the orbs to gather the most water?



Input

The first line contains one integer N (1 <= N <= 10^5), denoting the number of trees in the forest.
The second line contains the height of the trees separated by a space.

Output:

The output contains an integer number denoting the maximum area you can cover between two trees.

Sample Input/Output:

Sample Input 1	Sample Output 1
9 1 8 6 2 5 4 8 3 7	49
Sample Input 2	Sample Output 2
6	25

5 2 6 8 3 10	

#7

Problem Proposed/Prepared by: TAP

You are given a list consisting of N integers, you need to find all the distinct triplets present in the list which add up to zero.

A list is said to have a triplet (li[i], li[j], li[k]) with 0 sum if there exist three indices i, j, and k such that i!=j, j!=k and i!=k and li[i] + li[j] + li[k] = 0.

If no such triplet is present in the list, then the output should be “IMPOSSIBLE”.

Input

The first line contains an integer T (1 <= T <= 50), denoting the number of test cases.

For each test case, the first line will contain an integer N (1 <= N <= 10^3), denoting the number of elements of the list. In the next line, there will be N integers separated by space.

Output

For each test case, find the triplet/s that add up to 0. The values of the triplets can be printed in any order. For example, if a valid triplet is (1, 2, -3), then (2, -3, 1), (-3, 2, 1) etc. are also valid triplet. Also, the ordering of different triplets can be random i.e. if there are more than one valid triplets, you can print them in any order.

Sample Input/Output:

Input 1	Sample Output 1
1 -2 -3 2 -1 -1	Case 1: 1 2 -3 -1 -2 3 Case 2: IMPOSSIBLE Case 3: -1 -1 2
Input 2	Sample Output 2
	Case 1: -10 5 5

-5 2 0	Case 2: 0 0 0

Explanation:

Sample input 1:

For the first test case, 1+2-3 = 0, -1-2+3 = 0, hence, (1, 2, -3) and (-1, -2, 3) are two triplets that add up to zero and the output is 1 2 -3 and -1 -2 3.
For the second test case, since there are no such triplets, the output is "IMPOSSIBLE".
For the third test case, -1-1+2 = 0, hence, the only distinct triplet is (-1, -1, 2) and the output is -1 -1 2.

#8

Problem Proposed/Prepared by: ART

Suppose you are given **2** lists of tasks sorted in descending order based on their endtime. Your task is to create a new list containing only the odd numbered tasks(0 based indexing) in ascending order. Your algorithm should run in O(n) time complexity.

Input:
First line of the input file will denote the number **N** which represents the number of tasks in the first list. There will be N following lines of tasks in the following format: Start_time<space>End_time. Next line will be another integer M which represents the number of tasks in the second list. There will be M following lines of tasks in the same format.

Output:
The new task list which is sorted in ascending order based on the end times.

Note: The input must be read from a text file and the output should also be written in a separate text file. You can consider the input file name to be "input.txt".

Sample Input	Sample Output	Explanation
4 10 12 8 11 7 9 3 7 4 10 14 11 12 4 6 1 3	1 3 3 7 8 11 11 12	Odd positioned tasks of List-1: 8 11, 3 7 Odd positioned tasks of List-2: 11 12, 1 3 Comparing the tasks in both lists we can see that the smallest endtime is in list-2 which is 1 3, then 3 7 from list-1 and so on.

#9

Problem Proposed/Prepared by: FJP

You are given a sorted list of user activity timestamps. Each timestamp is an integer representing the number of seconds since the start of the day. It is observed that some of the timestamps are duplicates. Your goal is to print the values where each value will occur at most ***K*** Times.
Your solution should run in **O(n) time complexity.** In addition, you can only have one list to store the input values. You are not allowed to use any additional data structures (for example: array/list, dictionary/map, etc.) to store the input values.

Input:

The first line contains two integers N, K (1 <= N, K <= 10^5), denoting the length of the sorted list and K as the number of duplicates allowed.

The next line contains N integers $a_1, a_2, \ldots, a_n$ ($1 \leq a_i \leq 10^8$) separated by a space.

Output:
Print the timestamps such that for each value there are no more than K duplicates.

Note: The input must be read from a text file and the output should also be written in a separate text file. You can consider the input file name to be **"input.txt"**.

Input	Sample Output
0 150 150 150 200 300 300 300	100 100 150 150 200 300 300 400
2 5 5 5 5	1 2 2 2 5 5 5

#10

Problem Proposed/Prepared by: FZM

You are given a file "**inp3.txt**". The file contains the following info: In the first line there are two space separated integers **N** and **M (0 < N < 100000, 0 < M < 100000)**. In the second line there are **N** integers (absolute

value of each integer is less than 100000) separated by spaces and sorted in ascending order. In the third line there are **M** integers (absolute value of each integer is less than 100000) separated by spaces and sorted in ascending order.

Suppose you have two lists, the first list contains the **N** integers and the second list contains the **M** integers. You need to find all distinct integers which are present in both of the lists. Do it in **O(N+M)** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be some distinct integers separated by spaces in any order (the answer you were asked to find above) in the first and only line.

Sample Input File 1	Sample Output File 1
5 6 -1 2 2 3 4 2 2 3 3 5 60000	2 3
Sample Input File 2	Sample Output File 2
7 8 -1 2 3 4 5 6 7 2 3 5 6 7 8 9 10000	5 6 7 2 3

#11

Problem Proposed/Prepared by: PLN

You are given a file "**input3.txt**". The file contains the following info: In the first line there are two space separated integers **N** and **M (0 < N < 100000, 0 < M < 100000)**. In the second line there are **N** positive integers (each integer is less than 100000) separated by spaces and sorted in ascending order. In the third line there are **M** positive integers (each integer is less than 100000) separated by spaces and sorted in ascending order.

Suppose there are two teams. The first team has **N** players, their powers are the **N** integers. The second team has **M** players, their powers are the **M** integers. Some pairs of players are made such that in each pair, the first player is from the first team, the second player is from the second team, and the power of the first person is less than the power of the second person. Each player belongs to at most one pair. Find the maximum number of pairs you can make in this way. Do it in **O(N+M)** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.txt**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one integer (the answer you were asked to find above) in the first and only line.

Sample Input File 1	Sample Output File 1
5 6 1 2 2 3 4 2 2 3 3 5 60000	5
Sample Input File 2	Sample Output File 2
7 8 1 2 3 4 5 6 70000 2 3 5 6 7 8 9 10	6

#12

Problem Proposed/Prepared by: NZF

You are given a file "**inp3.txt**". The file contains the following info: In the first line there are three space separated integers **N, M** and **K (0 < N < 100000, 0 < M < 100000, 0 < K < 100000)**. In the second line there are **N** integers (absolute value of each integer is less than 100000) separated by spaces and sorted in ascending order. In the third line there are **M** integers (absolute value of each integer is less than 100000) separated by spaces and sorted in ascending order.

Suppose you have two lists **A** and **B**. **A** contains the **N** integers in the given order and **B** contains the **M** integers in the given order. You need to find any pair of two index numbers (0-based) **i** and **j** such that the sum of **A[i]** and **B[j]** is closest to **K**. Do it in **O(N+M)** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be two space separated integers **i** and **j** (the answer you were asked to find above) in the first and only line.

Sample Input File 1	Sample Output File 1
5 6 10 -1 2 2 3 4 2 2 3 3 5 60000	4 4
Sample Input File 2	Sample Output File 2
7 8 9999 -1 2 3 4 5 6 7 2 3 5 6 7 8 9 10000	0 7

#13

Problem Proposed/Prepared by: KKP

You are given a file "**input3.txt**". The file contains the following info: In the first line there are two space separated integers **N** and **K (0 < N < 100000, 0 < K < 100000)**. In the second line there are **N** positive integers (each integer is less than 100000) separated by spaces.

Suppose there is a list containing the **N** integers in the given order. You need to find any pair of two index numbers (0-based) **i** and **j** such that **i <= j** and the sum of all integers between index **i** and **j** in the list is closest to **K**. Do it in **O(N)** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.txt**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be two space separated integers **i** and **j** (the answer you were asked to find above) in the first and only line.

Sample Input File 1	Sample Output File 1
5 60 1 20 20 30 40000	2 3
Sample Input File 2	Sample Output File 2
7 80 1 20 30 40 5 6 70000	2 5

#14

Problem Proposed/Prepared by: ADJ

You are given a file "**inp3.txt**". The file contains the following info: In the first line there is an integer **N (0 < N < 1000)**. In the second line there are **N** space separated distinct integers (absolute value of each integer is less than 100000).

Suppose there is a list containing these **N** integers. Find all unique unordered triplets which can be made by taking three distinct numbers from the list such that they sum up to zero. In an unordered triplet, the order of the three integers does not matter. Do it in **O(N^2)** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be zero or more lines, each line containing three space separated integers (the answer you were asked to find above).

Sample Input File 1	Sample Output File 1
5 -1 2 4 8 -16000	
Sample Input File 2	Sample Output File 2

10 1 2 3 6 10 -3 -4 -5 -11 99999	1 2 -3 1 3 -4 -5 2 3 10 -11 1

#15

Problem Proposed/Prepared by: IZD

You are given a file "**input3.txt**". The file contains the following info: In the first line there are two space separated integers **N** and **K (0 < N < 100000, 0 < K < 100000)**. In the second line there are **N** positive integers (each integer is less than 100000) separated by spaces.

Suppose there is a list containing the **N** integers in the given order. You need to count how many pairs of two index numbers (0-based) **i** and **j** are there such that **i <= j** and the sum of all integers between index **i** and **j** in the list is less than **K**. Do it in **O(N)** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.txt**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one integer (the answer you were asked to find above) in the first and only line.

Sample Input File 1	Sample Output File 1
5 60 1 10 20 30 40000	8
Sample Input File 2	Sample Output File 2
7 80 1 20 30 40 5 6 70000	14

#18

Problem Proposed/Prepared by: MZU

Suppose you are given two sorted lists of integers **A** and **B**. Your task is to find the union of these two arrays. The union should include unique elements from both arrays in descending order.

Your algorithm should run in O(N+M) time complexity, where N and M are the lengths of A and B respectively.

Input: You are given a file "**input.txt**". There will be 3 lines in the sample input file. The first line will contain **N** and **M (1 <= N, M < 100000)**, the length of the two sorted lists. The second line contains **N** space-separated positive integers of list **A**. The third line contains

M space-separated positive integers of list **B**. All the integer values are less than 100000.

Output: A single line containing the elements of the union of the two arrays in descending order. You should show the output in an output file "**00000000.out**" (**replace the 0's with your ID number**).

Sample Input	Sample Output
5 6 1 2 2 3 4 2 2 3 3 5 6	6 5 4 3 2 1
9 8 2 4 5 5 8 9 9 12 15 1 1 5 6 7 12 12 20	20 15 12 9 8 7 6 5 4 2 1

#19

Problem Proposed/Prepared by: ARD

Suppose you are given two lists of integers **A** and **B**. Find if the integers of **B** is a subsequence of the integers of **A**.

A subsequence is defined as a sequence that can be derived from another sequence by deleting some or none of the elements without changing the order of the remaining elements.

Your algorithm should run in O(N+M) time complexity, where N and M are the lengths of A and B respectively.

Input: You are given a file "**inp.txt**". There will be 3 lines in the sample input file. The first line will contain **N** and **M (1 <= N, M < 100000)**, the length of the two sorted lists. The second line contains **N** space-separated positive integers of list **A**. The third line contains **M** space-separated positive integers of list **B**. All the integer values are less than 100000.

Output: Print "Yes" if B is a subsequence of A. Otherwise print "No". You should show the output in an output file "**00000000.out**" (**replace the 0's with your ID number**).

Sample Input	Sample Output
10 4 2 4 1 5 6 8 1 2 7 7 4 8 2 7	Yes
4 6 1 2 3 4 1 2 3 4 5 6	No

Previous Quiz Questions on Divide & Conquer

#1

Problem Proposed/Prepared by: ARD

You are given a file "**inp2.txt**". The file contains the following info: In the first and only line there are two integers **A** and **N (0 < A < 101, 0 < N < 1000000000)**.

Find the value of **(A^0 + A^1 + ... + A^N) % 107**. Here, x^y means x to the power of y.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one integer (the answer you were asked to find above) in the first and only line.

Sample Input File 1	Sample Output File 1
100 2	43
Sample Input File 2	Sample Output File 2
2 988244353	101

#2

Problem Proposed/Prepared by: KKS

You are given a file "**inp4.txt**". The file contains the following info: In the first line there is an integer **N (0 < N < 100000)**. In the second line there are **N** words separated by spaces. These words are made of alphabets only. Among the words, one word is given more than **N/2** times.

Find the most frequently occurred word among the given words. Do it in **O(N*log(N))** time. Do it using Divide and Conquer technique. Do not sort or modify the given words. Do not use builtin HashTables.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**"

(**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one string (the answer you were asked to find above) in the first and only line.

Sample Input File 1	Sample Output File 1
10 Rd Rd Bl Bl Bl Gr Gr Bl Bl Bl	Bl
Sample Input File 2	Sample Output File 2
7 Mx Mx Mx Mn Mx Mx Ma	Mx

#3

Problem Proposed/Prepared by: ANK

You are given a file "**inp4.txt**". The file contains the following info: In the first line there is an integer **N (0 < N < 100000)**. In the second line there are **N** integers separated by spaces.

Find an order of these **N** integers where all integers except for the **0**'s are in ascending order and all the **0**'s are at the end. Do it in **O(N*log(N))** time. Do it using Divide and Conquer technique. Do not use any builtin sorting method.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be **N** space separated integers (the answer you were asked to find above) in the first and only line.

Sample Input File 1	Sample Output File 1
7 0 1 4 1 7 9 0	1 1 4 7 9 0 0
Sample Input File 2	Sample Output File 2
8 1 0 0 -6 0 3 2 8	-6 1 2 3 8 0 0 0

#4

Problem Proposed/Prepared by: FHZ

You are given a file "**inp4.txt**". The file contains the following info: In the first line there is an integer **N (0 < N < 100000)**. In the second line there are **N** integers (absolute value of each is less than 100000) separated by spaces. Among all these integers, all the odd integers (if there are any) appear at the start and all the even integers (if there are any) appear at the end.

First prepare a list or array containing these **N** integers in the given order. Then find how many odd integers and how many even integers are there among all these integers in **O(log(N))** time. Do it using Divide and Conquer technique. Do not sort or modify the list or array.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be two space separated integers (the answer you were asked to find above) in the first and only line.

Sample Input File 1	Sample Output File 1
8 5 9 21 49 3 14 76 2	5 3
Sample Input File 2	Sample Output File 2
5 15 1 17 9 33	5 0

#5

Problem Proposed/Prepared by: KKP

You are given a file "**inp4.txt**". The file contains the following info: In the first and only line there are two integers **A** and **N (0 < A < 107, 0 < N < 1000000000)**.

Find the remainder when (**A** to the power of **N**) is divided by **107**. Do it in **O(log(N))** time. Do it using Divide and Conquer technique. Do not use any builtin function except for file I/O and string processing.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one integer (the answer you were asked to find above) in the first and only line.

Sample Input File 1	Sample Output File 1
99 1	99
Sample Input File 2	Sample Output File 2
99 2	64
Sample Input File 2	Sample Output File 2
106 988244353	106

#6

Problem Proposed/Prepared by: ARD

You are given a file "**inp2.txt**". The file contains the following info: In the first line there is an integer **N (0 < N < 100000)**. In the second line, there are **N** integers (the absolute value of each integer is less than 100000) separated by spaces and sorted in ascending order.

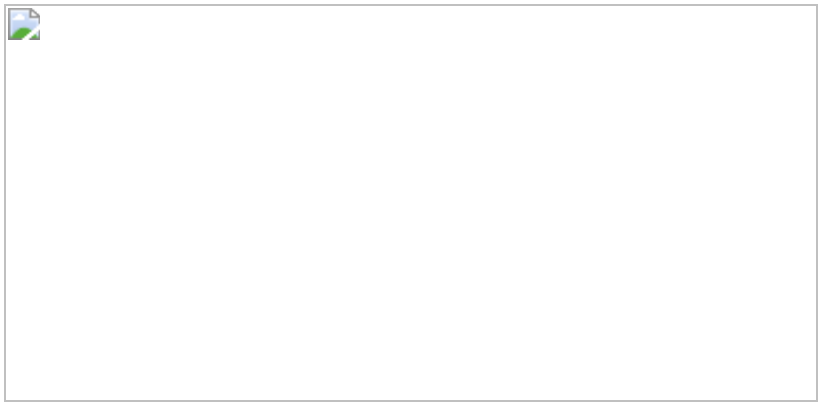
Find an order of these **N** integers such that, if these integers are inserted into a Binary Search Tree one by one in that order then at the end the height of the Binary Search Tree is minimum. If there are multiple such orders then find any of them. Do it in **O(N*log(N))**. Do it using the Divide and Conquer approach.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file, there should be **N** space-separated integers (the answer you were asked to find above) in the first and only line.

Sample Input File 1	Sample Output File 1
11 -4 -2 -1 1 2 4 8 16 32 64 128	4 -1 -4 -2 1 2 32 8 16 64 128
Sample Input File 2	Sample Output File 2
4 1 2 3 4	2 3 4 1

Note: Binary Search Tree is a Binary Tree where, for each node, all the values in the left subtree of that node are less than the value of that node, and all the values in the right subtree of that node are greater than the value of that node.



#7

Problem Proposed/Prepared by: ARD

There are **N** integers **A[0]**, **A[1]**, … **A[N-1]**. Find any pair of indexes **(i,j)** such that **0 <= i < j < N** and the value of **A[i]^2 + A[j]^3** is the maximum possible.

Input
The first line contains one integer **N** where **0 < N < 10^5**. The second line contains **N** space separated integers, the values of **A[i]**, each of which is in the range **[-10^5, 10^5]**.

Output:
Two space separated integers, **i** and **j**.

Sample Input/Output:

Sample Input 1	Sample Output 1
5 9 6 5 8 2	0 3
Sample Input 2	Sample Output 2
8 5 10 4 -3 1 6 -10 2	0 1
Sample Input 3	Sample Output 3
7 -5 -2 -6 -7 -1 8 2	3 5

#8

Problem Proposed/Prepared by: KKP

You are on a long drive trip. You have **N** music CDs with you. Each CD has an integer value, **P,** assigned to it, representing your level of preference for that CD. Now, you want to choose a contiguous segment of music CDs such that the sum of the preference levels is maximum.

Come up with an O(n log n) algorithm.

Input:

The first line contains an integer N ($1 <= N <= 10^5$), denoting the total number of music CDs you have brought.

The second line has N integers $P_1,P_2,\ldots,P_N$ ($-10^9 <= P_i <= 10^9$)- denotes the favouriteness level of the i-th CD.

Output:

Print one integer: which denotes the sum of the maximum preference you can have by choosing a contiguous segment of music CDs.

Sample Input/Output:

Sample Input 1	Sample Output 1
8 -1 3 -2 5 3 -5 2 2	9
Sample Input 2	Sample Output 2

7 6 -1 -2 1 3 -4 1	7
Sample Input 3	Sample Output 3
4 -1 -4 -3 -2	0
Sample Input 4	Sample Output 4
3 10 10 5	25

#9

Problem Proposed/Prepared by: NFH

Given an array of integers, find the contiguous subarray with the largest sum and print the starting index and ending index of that subarray. Your program should not take more than O(n log n) time.

Input:

The first line denotes **N (1 <= N <= 10^5)**, the length of the array. The next line contains **N** values of the array.

Output:

Two integers **S** and **E** represent the starting and ending indices **(1-indexed)** of the contiguous subarray with the maximum sum.

Sample Input/Output:

Sample Input 1	Sample Output 1
9 -2 1 -3 4 -1 2 1 -5 4	4 7
Sample Input 2	Sample Output 2
5 1 3 4 1 2	1 5

Explanation:

In sample input 1, the array is [-2,1,-3,**4,-1,2,1**,-5,4]. The subarray shown in bold has the maximum sum. So, the starting index of the subarray is 4, and the ending index of the subarray is 7.
In sample input 2, the array is **[1,3,4,1,2]**. The subarray shown in bold (in this case, the entire array) has the maximum sum. So, the starting

index of the subarray is 1, and the ending index of the subarray is 5.

#10

Problem Proposed/Prepared by: SAK

Given an input binary number, find the number of **1s** in the binary number **using a divide and conquer implementation**.

Sample Input 1	**Sample Output 1**
101101	4
Sample Input 2	**Sample Output 2**
1110	3
Sample Input 3	**Sample Output 3**
000	0

#11

You are given a file "**inp4.txt**". The file contains the following info: In the first line there is an integer **N (1 < N < 500000)**. In the second line there are **N** integers, each between **1** and **500000**.

There is a list containing these **N** integers in the given order, the **x**'th item in this list is called **arr[x]**. Find the number of pairs of indexes **(i, j)** such that **i < j** and **arr[i] > 2 * arr[j]**. Solve the problem in **O(N*log(N))** time using a Divide and Conquer approach.

Sample Input File 1	Sample Output 1
5 4 2 1 5 3	1
Sample Input File 2	Sample Output 2
4 1 4 3 10	0
Sample Input File 3	Sample Output 3
11 9 6 400000 2 12 5 300000 7 1 15 13	19
Sample Input File 4	Sample Output 4
4 23 11 5 2	6

#12

You are given a file "**inp4.txt**". The file contains the following info: In the first line there is an integer **N (0 < N < 5000)**. In the second line

there are **N** distinct integers, in the third line there are **N** distinct integers, all integers are between **1** and **500000**.

There is a Binary Tree such that its pre-order traversal is the same as the order of the **N** integers in the second line and its post-order traversal is the same as the order of the **N** integers in the third line. Find the in-order traversal of any such Binary Tree. Solve the problem in **O(N^2)** time using a Divide and Conquer approach.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one line containing the **N** integers such that their order is the same as the in-order traversal you are asked to find above.

Sample Input File 1	Sample Output File 1
5 3 9 20 15 7 9 15 7 20 3	9 3 15 20 7
Sample Input File 2	Sample Output File 2
10 1 2 3 4 5 6 7 8 9 500000 500000 9 8 7 6 5 4 3 2 1	1 2 3 4 5 6 7 8 9 500000

#13

You are given a file "**inp4.txt**". The file contains the following info: In the first line there is an integer **N (0 < N < 5000)**. In the second line there are **N** distinct integers, in the third line there are **N** distinct integers, all integers are between **1** and **500000**.

There is a Binary Tree such that its pre-order traversal is the same as the order of the **N** integers in the second line and its in-order traversal is the same as the order of the **N** integers in the third line. Find the post-order traversal of that Binary Tree. Solve the problem in **O(N^2)** time using a Divide and Conquer approach.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one line containing the **N** integers such that their order is the same as the post-order traversal you are asked to find above.

Sample Input File 1	Sample Output File 1
5	9 15 7 20 3

3 9 20 15 7 9 3 15 20 7	
Sample Input File 2	Sample Output File 2
10 1 2 3 4 5 6 7 8 9 500000 1 2 3 4 5 6 7 8 9 500000	500000 9 8 7 6 5 4 3 2 1

#14

You are given a file "**inp4.txt**". The file contains the following info: In the first line there is an integer **N (0 < N < 5000)**. In the second line there are **N** distinct integers, in the third line there are **N** distinct integers, all integers are between **1** and **500000**.

There is a Binary Tree such that its in-order traversal is the same as the order of the **N** integers in the second line and its post-order traversal is the same as the order of the **N** integers in the third line. Find the pre-order traversal of that Binary Tree. Solve the problem in **O(N^2)** time using a Divide and Conquer approach.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one line containing the **N** integers such that their order is the same as the pre-order traversal you are asked to find above.

Sample Input File 1	Sample Output File 1
5 9 3 15 20 7 9 15 7 20 3	3 9 20 15 7
Sample Input File 2	Sample Output File 2
10 1 2 3 4 5 6 7 8 9 500000 500000 9 8 7 6 5 4 3 2 1	1 2 3 4 5 6 7 8 9 500000

#15

You are given a file "**txt4.inp**". The file contains the following info: In the first line there are four integers **N, A, B, C (0 < N < 500000, 0 < A < 500000, 0 < B < 500000, 0 < C < 50000000000000000)**.

There is a 2D array **arr** of size **N** by **N** such that **arr[x][y] = A*x^2 + B*y^2** (**A** times **x** squared plus **B** times **y** squared). The index numbers in the array are 0-based. Find how many numbers in this array are greater than **C**, in **O(N*log(N))** time using a Divide and Conquer approach.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.txt**"

(**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one integer, the length of the longest subarray you are asked to find above.

Sample Input File 1	Sample Output File 1
4 1 2 18	3
Sample Input File 2	Sample Output File 2
10 1 2 100	35
Sample Input File 3	Sample Output File 3
400000 1 1 400000	159999685206

Previous Quiz Questions on Graph Traversal (BFS, DFS) and their Applications

#1

Problem Proposed/Prepared by: ARD

There are **N** integers: **0, 1, … (N-1)**. Also, there are **M** rules: **(X[0], Y[0]), (X[1], Y[1]), … (X[M-1], Y[M-1])**. Find the lexicographically largest permutation of the **N** integers such that for each of the rules **(X[i], Y[i])**, **X[i]** comes before **Y[i]** in the permutation. If there is no such permutation then detect that.

Input
The first line contains two integers **N** and **M**, where **0 < N < 10^5** and **0 < M < 10^5**. The second line contains **M** space-separated integers, the values of **X[i]**. The third line contains **M** space-separated integers, the values of **Y[i]**. Each of the values **X[i]** and **Y[i]** are in the range **[0, N-1]**.

Output:
If there is a permutation that satisfies the rules mentioned above then print all the **N** integers in the order they appear in the permutation, each integer separated by spaces. If no permutation satisfies the rules mentioned above then print **-1** instead.

Sample Input/Output:

Sample Input 1	Sample Output 1
5 3 3 1 4 1 2 0	4 3 1 2 0
Sample Input 2	Sample Output 2
6 6 1 2 4 4 5 0 2 3 3 5 0 4	-1
Sample Input 3	Sample Output 3

8 10 1 1 2 2 2 4 4 6 5 7 2 4 4 5 3 6 5 5 3 0	7 1 2 4 6 5 3 0

#2

Problem Proposed/Prepared by: KKP

The Crows have invented a new dictionary of N words. The words are stored in an order which is different from our regular Latin lexicographic order.

Now you have to find the order of the alphabet that satisfies the lexicographic order of the new dictionary. If there are multiple valid orders you may print any one of those.

Input
The first line contains an integer N (1 <= N <= 1000) - denotes the number of words in the dictionary.

The next N line contains a string S (1 <= |S| <= 100). Each word consists of lowercase Latin letters (‘a’-‘z’).

Output
Find out the order of the alphabets that satisfy the sorting order of the words in the given dictionary.

Sample Input/Output:

Sample Input 1	Sample Output 1
9 error tooth tot teeth their there thi tie hit	o e t h i r [This is the only valid order]
Sample Input 2	Sample Output 2
3 eat tea ate	e t a [This is the only valid order]

#3

Problem Proposed/Prepared by: FJP

You are given a maze represented as a 2D grid of size RxH. Each cell in the grid can be an empty space, denoted by '.', a wall 'W', a starting point 'S', or a destination 'D'. There are two players in the jungle, starting from different points, and they aim to reach a single destination point. The objective is to determine which player will reach the

destination first. If both players reach the destination at the same time, the result should indicate a tie.

The paths must be the shortest possible in terms of the number of cells traversed. You can only move horizontally or vertically to an adjacent cell.

Input:

- The first line contains two integers R and H (1 <= R, H <= 100) — the number of rows and columns respectively in the grid.
- The second line indicates the coordinates of the starting cells for Player 1.
- The third line indicates the coordinates of the starting cells for Player 2.
- The next R line will contain H characters. Each character represents the status of a cell as follows.
  - '.': Empty Cell → You can move here.
  - 'D': Destination
  - 'W': Cell with an obstacle → You can't move to this cell.

Output:

- A string indicating which player reaches the destination first ("Player 1", "Player 2"), or "Tie" if both reach at the same time.

Input	Output
5 5 2 0 4 0 ..w.. .w.w. ..w.D .w.w.	Player 2
5 5 0 0 4 0 ....w. ..w.D .w.w.	Tie

#4

Problem Proposed/Prepared by: KKP

You are given an undirected and unweighted graph with ***N*** cities and ***N-1*** roads. Every city has a value, ***P,*** which denotes the amount of Gold you can get if you visit the city. You can collect the gold only once.

You are starting your journey from city 1. Now, you have to find a city, ***T,*** such that, if you visit from city 1 to T, you can collect the maximum amount of gold.

Input
The given map will be an undirected and unweighted graph.

The first line contains two integers N and M (1 <= N <= 1000) – the number of cities and the total number of roads.

In the next line, there will be N values, P (1 <= P_i <= 1000)- which denotes the number of gold you can collect going to the city.

The next M lines will contain two integers u_i, v_i (1 <= u_i, v_i <= N) – denoting there is a bidirectional road between u_i and v_i.

Output
Print the city number and maximum amount of gold you can get starting from city 1.

Sample Input/Output:

Sample Input 1	Sample Output 1
7 6 5 10 2 3 6 10 7 1 2 1 5 2 3 2 4 5 6 5 7	21
Sample Input 2	Sample Output 2
7 6 1 1 1 1 1 3 2 1 2 2 3 3 4 1 5 5 6 5 7	5

#5

Problem Proposed/Prepared by: PLN

In a small village, Ella the painter loves to add color to old photos. One day, she finds a faded photo represented by a grid, where 0 means gray (can't be colored) and 1 means black. Ella starts coloring at a specific point (sr, sc) with her chosen color. She can only move diagonally (top-left, top-right, bottom-left, bottom-right) from one cell to another.

Starting from the image[sr][sc], she spreads her color to any diagonally connected cells that share the same initial color. Ella can color a cell if and only if that cell contains black color. Now help Ella by simulating this process: replace the color of all the relevant pixels with her chosen color and return the newly colored photo.

Input:

1. The first line of the input contains 3 integers sr, sc, and color (0<sc, sr, color<10^3).
2. The next line contains 2 integers n and m representing the photo size (0<n,m<10^3).
3. The following n lines contain m integers representing each pixel's color.

Output:
Return the colored image

Sample Input File 1	Sample Output File 1
1 1 2 3 3 0 1 0 1 1 1 0 1 0	0 1 0 1 2 1 0 1 0
Sample Input File 2	Sample Output File 2
1 1 2 3 3 1 1 1 1 1 0 1 0 1	2 1 2 1 2 0 2 0 2

#6

Problem Proposed/Prepared by:

You are given a file "**txt5.inp**". The file contains the following info: In the first line there are two integers **N, M (0 < N < 500000, 0 < M < 500000)**. In the second line and in the third line there are **M** integers each, all integers in these two lines are between **1** and **N**.

There is a directed unweighted graph with **N** nodes and **M** edges. The nodes are numbered from **1** to **N**. The **i**'th edge of this graph is from the **i**'th node in the second line to the **i**'th node in the third line, there is no self loop or multi edge. A simple cycle is a path which starts and ends at the same node such that no edge is present more than once in that path. Find if there is any simple cycle in this graph, and if there is then find any such cycle. Do everything in **O(N+M)** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.txt**"

(**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one line. If there is no simple cycle in the graph then this line contains **0**. Otherwise, this line contains the nodes of one such cycle in the order of the path.

Sample Input File 1	Sample Output File 1
6 4 6 2 3 5 2 3 4 1	0
Sample Input File 2	Sample Output File 2
12 12 1 1 1 2 2 2 3 3 7 4 12 11 2 3 4 5 6 7 8 9 1 12 5 10	1 2 7 1

#7

Problem Proposed/Prepared by:

You are given a file "**inp5.txt**". The file contains the following info: In the first line there are three integers **N, M, K (0 < N < 5000, 0 < M < 5000, 0 < K < 500000)**. In each of the next **N** lines, there are **M** integers, each integer is between **1** and **500000**.

The **N** lines starting from the second line represents a 2D array **arr** of size **N** by **M** such that **arr[x][y] = y**'th integer in the **x**'th line. There is a grid of size **N** by **M**, where the height of the cell at position **(x,y)** is **arr[x][y]**. The index numbers in both the array and the grid are **1**-based. A jump (both way) between two adjacent (**up or down or left or right**) cells is possible if and only if the absolute difference between their heights is less than or equal to **K**. Find if there is a path from the cell at position **(1,1)** to the cell at position **(N,M)**, and if there is then find any such path. Do everything in **O(N*M)** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one line. If there is no path following the above rules then this line contains **0**. Otherwise this line contains the cells representing any path following the above rules, each cell is given in the format "**(x,y)**", adjacent cells are separated by spaces, there is no space between the coordinates of a cell.

Sample Input File 1	Sample Output File 1
3 3 2 1 2 2 3 8 2 5 7 5	(1,1) (2,1) (3,1) (3,2) (3,3)

Sample Input File 2	Sample Output File 2
1 10 2 1 2 3 4 5 6 7 8 7 11	0

#8

Problem Proposed/Prepared by:

You are given a file "**txt5.inp**". The file contains the following info: In the first line there are two integers **N, M (0 < N < 500000, 0 < M < 500000)**. In the second line and in the third line there are **M** integers each, all integers in these two lines are between **1** and **N**.

There is an undirected unweighted graph with **N** nodes and **M** edges. The nodes are numbered from **1** to **N**. The **i**'th edge of this graph is between the **i**'th node in the second line and the **i**'th node in the third line, there is no self loop or multi edge. Find the minimum number of edges (and any valid set of such edges) that you need to remove from this graph such that after the modification there is no cycle in this graph. Do everything in **O(N+M)** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.txt**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be three lines. The first line contains the minimum number of edges. The second and third lines contain the description of the edges you are asked to find above, the **i**'th edge is between the **i**'th node in the second line and the **i**'th node in the third line. If the first line contains 0 then the other lines are empty.

Sample Input File 1	Sample Output File 1
6 4 6 2 3 5 2 3 4 1	0
Sample Input File 2	**Sample Output File 2**
12 12 1 1 1 2 2 2 3 3 7 4 12 11 2 3 4 5 6 7 8 9 1 12 5 10	2 1 1 2 7

#9

Problem Proposed/Prepared by:

You are given a file "**txt5.inp**". The file contains the following info: In the first line there are two integers **N, M (0 < N < 500000, 0 < M < 500000)**. In the second line and in the third line there are **M** integers each, all integers in these two lines are between **1** and **N**.

There is an undirected unweighted graph with **N** nodes and **M** edges. The nodes are numbered from **1** to **N**. The **i**'th edge of this graph is between the

i'th node in the second line and the **i**'th node in the third line, there is no self loop or multi edge. Find the minimum number of edges (and any valid set of such edges) that you need to add to this graph such that after the modification all pairs of nodes are connected by at least one path. Do everything in **O(N+M)** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.txt**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be three lines. The first line contains the minimum number of edges. The second and third lines contain the description of the edges you are asked to find above, the **i**'th edge is between the **i**'th node in the second line and the **i**'th node in the third line. If the first line contains 0 then the other lines are empty.

Sample Input File 1	Sample Output File 1
6 5 6 2 3 5 1 2 3 4 1 2	0
Sample Input File 2	**Sample Output File 2**
13 12 1 1 1 2 2 2 3 3 7 4 12 11 2 3 4 5 6 7 8 9 1 12 5 10	2 1 13 10 2

#10

Problem Proposed/Prepared by:

You are given a file "**inp5.txt**". The file contains the following info: In the first line there are two integers **N, M (0 < N < 500000, 0 < M < 500000)**. In the second line and in the third line there are **M** integers each, all integers in these two lines are between **1** and **N**.

There is an undirected unweighted graph with **N** nodes and **M** edges. The nodes are numbered from **1** to **N**. The **i**'th edge of this graph is between the **i**'th node in the second line and the **i**'th node in the third line, there is no self loop or multi edge. Find how many nodes and how many edges are reachable from the node numbered **N**, in **O(N+M)** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one line containing two integers, the number of reachable nodes followed by the number of reachable edges, which you were asked to find above.

Sample Input File 1	Sample Output File 1
6 4	4 3

6 2 3 5 2 3 4 1	
Sample Input File 2	Sample Output File 2
12 12 1 1 1 2 2 2 3 3 7 4 12 11 2 3 4 5 6 7 8 9 1 12 5 10	10 11

#11

Problem Proposed/Prepared by:

You are given a file "**txt5.inp**". The file contains the following info: In the first line there are two integers **N, M (0 < N < 500000, 0 < M < 500000)**. In the second line and in the third line there are **M** integers each, all integers in these two lines are between **1** and **N**.

There is an undirected unweighted graph with **N** nodes and **M** edges. The nodes are numbered from **1** to **N**. The **i**'th edge of this graph is between the **i**'th node in the second line and the **i**'th node in the third line, there is no self loop or multi edge. There are two colors: **A** and **B**. Find if it is possible to assign each node any of these two colors such that no two adjacent nodes have the same color, and if possible then find any such coloring. Do everything in **O(N+M)** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.txt**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one line. If it is impossible to color each node following the rules mentioned above then this line contains **X**. Otherwise, this line contains the color of each node from **1** to **N**.

Sample Input File 1	Sample Output File 1
6 4 6 2 3 5 2 3 4 1	A A B A B B
Sample Input File 2	Sample Output File 2
12 12 1 1 1 2 2 2 3 3 7 4 12 11 2 3 4 5 6 7 8 9 1 12 5 10	X

Previous Quiz Questions on Dijkstra

#1

Problem Proposed/Prepared by: ARD

There is a directed weighted graph with **N** nodes: **0, 1, … (N-1)**. The graph has **M** edges: **(X[0], Y[0], Z[0]), (X[1], Y[1], Z[1]), … (X[M-1], Y[M-1], Z[M-1])**, where **(X[i], Y[i], Z[i])** means there is an edge from node **X[i]** to

node **Y[i]** with weight **Z[i]**. Find any second shortest path from node **0** to node **1** in the graph. The graph has no nodes with a self loop. The graph has no multiple edges between the same pair of nodes.
Note that the length of the second shortest path might be equal to the length of the shortest path in some graphs, in that case any path with that length can be called the shortest path or the second shortest path. Two paths are different if the sequence of nodes in the paths are different.

Input

The first line contains two integers **N** and **M**, where **0 < N < 10^5** and **0 < M < 10^5**. The second line contains **M** space separated integers, the values of **X[i]**. The third line contains **M** space separated integers, the values of **Y[i]**. The fourth line contains **M** space separated integers, the values of **Z[i]**. Each of the values **X[i], Y[i], Z[i]** are in range **[0, N-1]**.

Output:

If there is no second shortest path from node **0** to node **1** then print **-1**. Otherwise print the sequence of nodes in any second shortest path starting from node **0** ending to node **1**.

Sample Input/Output:

Sample Input 1	Sample Output 1
5 4 0 2 3 1 2 3 1 4 1 1 1 1	-1
Sample Input 2	Sample Output 2
5 3 0 2 4 2 3 1 1 1 1	-1
Sample Input 3	Sample Output 3
11 9 0 0 0 2 10 4 8 9 7 2 10 4 1 1 1 9 7 8 1 2 3 1 1 1 1 1 10	0 10 1
Sample Input 4	Sample Output 4
11 9 0 0 0 2 10 4 8 9 7 2 10 4 1 1 1 9 7 8 1 1 3 1 1 1 1 1 10	0 10 1
Sample Input 5	Sample Output 5
11 9 0 0 0 2 10 4 8 9 7 2 10 4 1 1 1 9 7 8 1 1 3 1 1 1 1 1 10	0 2 1
Sample Input 6	Sample Output 6

6 6 0 2 3 4 5 4 2 3 4 5 2 1 1 1 1 1 1 1	0 2 3 4 5 2 3 4 1

#2

Problem Proposed/Prepared by: ANK

You are an owner of a transportation company and you ordered your employees to use the shortest route possible from source to destination so that the vehicles consume less fuel. The vehicle is in source s and the destination is in D. The graph is a directed and weighted graph. Now find the shortest route possible.

Input

The first line contains two integers N ($1 \leq N \leq 10000$) and M ($1 \leq M \leq 100000$) separated by a space, denoting the number of nodes and edges in the graph, respectively.

The next M lines each contain three integers, u, v (1 <= u, v <= N), and w (1 <= w <= 100) denoting an edge from node u to node v with weight w.

The last line contains two integers S,D (1 <= S,D <= N) which denotes the starting node of source and the destination node respectively.

Output

Output the cost of the shortest path and the path to destination from source. If there is no such route to the destination, print “Impossible”.

	Sample Output 1
	Time 3 Route 1->2->3
	Sample Output 2
	Impossible
	Sample Output 3
	Time 3 Route 5->1->2->8->6

#3

Problem Proposed/Prepared by: ATY

Siri had drawn an undirected graph of *n* vertices numbered from 0 to *n* - 1 and *m* edges between them. Each edge of the graph is weighted, each weight is a positive integer. However, some of the weights were erased. So now she has to reassign positive integer weight to each of the edges which weights were erased, so that the length of the shortest path between vertices *s* and *d* in the resulting graph is exactly *L.*

Input

The first line contains five integers *n*, *m*, *L*, *s*, *d*

n— the number of vertices, m- number of edges, L- the desired cost of the shortest path, s- starting vertex and d-ending vertex respectively.

Then, *m* lines describing the edges of the graph. Each contains three integers, *u*, *v*, *w.*

u and v denote the endpoints of the edge and wi denotes its weight. If wi is equal to 0 then the weight of the corresponding edge was erased.

It is guaranteed that there is at most one edge between any pair of vertices.

OUTPUT

Print "NO" if it's not possible to assign any weights in a required way.

Otherwise, print "YES" in the first line. Next *m* lines should contain the edges of the resulting graph, with weights assigned to edges which weights were erased.

The edges of the new graph must coincide with the ones in the graph from the input. The weights that were not erased must remain unchanged whereas the new weights can be any positive integer.

Sample Input	Sample Output
5 5 13 0 4 0 1 5 2 1 2 3 2 3 1 4 0	YES 0 1 5 2 1 2 3 2 3 1 4 8

4 3 4	4 3 4
4 5 7 1 3 1 2 5 1 4 8 2 4 7 2 3 0 3 4 6	YES 1 2 5 1 4 8 2 4 7 2 3 2 3 4 6

Explanation:

In the first sample case, there is only one missing edge weight. Placing the weight of 8 gives a shortest path from 0 to 4 of length 13.

#4

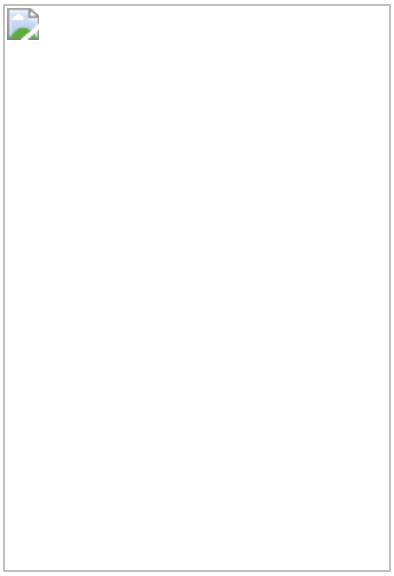
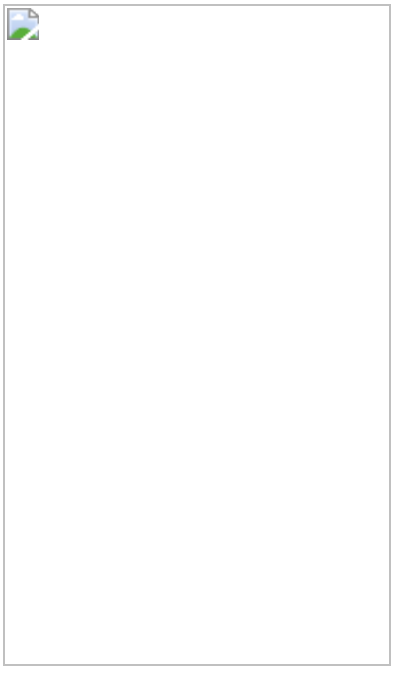
Problem Proposed/Prepared by: HAS

You're given a map of a fantasy kingdom represented as a weighted undirected graph. Each node represents a city, and the weight of an edge between two nodes represents the travel time between those cities. Your job is to find the quickest route for a knight to travel from the capital (city 1) to a remote village (city N).

Input:
The first line consists of two integers N (number of cities) and M (number of edges) separated by a space.The next M lines describe the edges. Each line contains three integers u, v, and w, representing a one-way street from city u to city v with travel time w.

Output:
Print the villages list, containing similar minimum travel time from the capital (city 1). Also, -1 if there is no similar minimum travel time for multiple villages.

Sample Input	Sample Output	Sample Graph

5 8 1 2 2 1 3 4 1 4 1 2 3 3 2 5 5 3 4 2 4 5 1 3 5 1	2 5	
5 7 1 2 3 1 3 4 2 4 1 2 3 3 2 5 5 3 4 2 4 5 1	3 4	
1 2 3 1 3 4 2 4 5 2 3 3 2 5 5 3 4 2 4 5 1	-1	

For sample 1 , After completing the dijkstra algorithm , 2 & 5 has similar travel time 2.
For sample 2 , After completing the dijkstra algorithm , 3 & 4 has similar travel time 4.
For sample 1 , After completing the dijkstra algorithm , No minimum similar travel time for villages.

#5

Problem Proposed/Prepared by: SAK

You are given an undirected weighted graph of **N** vertices (labeled as 0, 1, 2, …, N-1) and **M** edges. Each edge has a weight **p** that denotes the probability of successfully going from one vertex to the other through that edge. You will be given a **start** vertex and an **end** vertex, and you have to calculate what the maximum probability is of successfully traveling from the **start** vertex to the **end** vertex. If there is no path from the **start** vertex to the **end** vertex, then return **0.0**.

Input
The first line contains the number of vertices **N (2 <= N <= 50)** followed by the number of edges **M (0 <= M <= N(N-1)/2)**. Then, each of the following **M** lines contains three integers **u**, **v (0 <= u,v <= N-1, u!=v)**, **p (0 <= p <= 1)** – denoting an edge between vertex **u** and **v** with weight **p**. The last line contains two integers that denote the **start (0 <= start <= N-1)** vertex and the **end (0 <= end <= N-1)** vertex respectively.

Output
A single floating point value that denotes the maximum probability of traveling to the **end** vertex from the **start** vertex.

Sample Input 1	**Sample Output 1**	**Illustration 1**	**Explanation 1**
3 3 0 1 0.5 0 2 0.22 1 2 0.5 0 2	0.25		There are two paths from vertex 0 to vertex 2. **Path 0,2** has a probability of **0.22. Path 0,1,2** has a probability of **0.5*0.5=0.25** which is the higher probability.
Sample Input 2	**Sample Output 2**	**Illustration 2**	**Explanation 2**
3 2 1 0 0.3 1 2	0.0		There is no path from vertex 1 to vertex 2, therefore probability is **0.0**.

#6

Problem Proposed/Prepared by: SBDK

You are given a file "**inp5.txt**". The file contains the following info: In the first line there are four integers **N, M, S (0 < N < 500000, 0 < M < 500000, 1 <= S <= N)**. In the second line and in the third line there are **M** integers each, all integers in these two lines are between **1** and **N**. In the fourth line, there are **M** integers, all integers in this line are between **1** and **500000**.

There is a directed weighted graph with **N** nodes and **M** edges. The nodes are numbered from **1** to **N**. The **i**'th edge of this graph is from the **i**'th node in the second line to the **i**'th node in the third line, their weight is the **i**'th value in the fourth line, there is no self loop or multi edge. The total cost of a path is the sum of the weights of the edges in that path. A cycle is a path which starts and ends at the same node. You need to find out the minimum weighted cycle (containing at least two nodes) starting from node **S**. Solve the problem in **O((N+M)*log(N+M))** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one integer. If there is no cycle satisfying the above conditions then this integer is **-1**. Otherwise this integer is the minimum weighted cycle starting from **S**.

Sample Input File 1	Sample Output File 1
5 6 2 2 3 1 2 4 5 3 1 2 4 5 2 3 4 6 7 2 1	10
Sample Input File 2	Sample Output File 2
6 7 2 2 3 1 6 2 4 2 3 1 6 3 4 5 5 3 4 2 2 7 2 1	-1

#7

Problem Proposed/Prepared by: KKP

You are given a file "**inp5.txt**". The file contains the following info: In the first line there are six integers **N, M, S1, S2, S3, D (0 < N < 500000, 0 < M < 500000, 1 <= S1 <= N, 1 <= S2 <= N, 1 <= S3 <= N, 1 <= D <= N)**. In the second line and in the third line there are **M** integers each, all integers in these two lines are between **1** and **N**. In the fourth line, there are **M** integers, all integers in this line are between **1** and **500000**.

There is an undirected weighted graph with **N** nodes and **M** edges. The nodes are numbered from **1** to **N**. The **i**'th edge of this graph is between the **i**'th node in the second line and the **i**'th node in the third line, their weight (time required to cross the edge) is the **i**'th value in the fourth line, there is no self loop or multi edge. The total time of a path is the sum

of the weights of the edges in that path. Here, 3 persons start from **S1**, **S2**, and **S3** simultaneously and want to reach the destination **D**. You need to find the **minimum** time needed for **all** three persons to reach the destination **D**. Solve the problem in **O((N+M)*log(N+M))** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file, there should be only one integer. This integer is the **minimum** time needed for **all** three persons to reach the destination.

Sample Input File 1	Sample Output File 1
6 5 1 2 6 5 6 2 3 4 5 3 3 4 1 4 1 2 4 8 9	17
Sample Input File 2	Sample Output File 2
8 10 6 7 8 1 1 1 1 2 2 2 3 1 8 7 2 3 4 5 6 7 8 6 2 4 6 6 5 9 1 8 9 9 2 1	8

[Solve the same problem for directed graph]

#8

Problem Proposed/Prepared by: SBDK

You are given a file "**inp5.txt**". The file contains the following info: In the first line there are four integers **N, M, S, D (0 < N < 500000, 0 < M < 500000, 1 <= S <= N, 1 <= D <= N)**. In the second line and in the third line there are **M** integers each, all integers in these two lines are between **1** and **N**. In the fourth line, there are **M** integers, all integers in this line are between **1** and **500000**. In the fifth line, there are **M** integers, all integers in this line are between **1** and **2**.

There is an undirected weighted graph with **N** nodes and **M** edges. The nodes are numbered from **1** to **N**. The **i**'th edge of this graph is between the **i**'th node in the second line and the **i**'th node in the third line, their weight is the **i**'th value in the fourth line and their color is the **i**'th value in the fifth line, there is no self loop or multi edge. The cost of a path is the sum of the weights of the edges in that path. You need to find the cost of a path, if there is any, from node **S** to node **D** with the minimum cost satisfying the following rule: once you use an edge of color **2**, you can never use any edge of color **1** again. Solve the problem in **O((N+M)*log(N+M))** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**"

(**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one integer. If there is no path satisfying the above rules then this integer is **-1**. Otherwise this integer is the cost of the path you are asked to find above.

Sample Input File 1	Sample Output File 1
6 5 1 5 6 2 3 4 5 2 3 4 1 4 1 2 4 8 9 1 1 2 2 1	-1
Sample Input File 2	**Sample Output File 2**
12 12 2 11 1 1 1 2 2 2 3 3 7 4 11 12 2 3 4 5 6 7 8 9 1 11 5 10 1 2 4 8 1 2 4 8 1 2 8 16 1 2 1 2 1 2 1 2 1 2 2 2	7

#9

Problem Proposed/Prepared by: KKP

You are given a file "**txt5.inp**". The file contains the following info: In the first line there are four integers **N, M, S, D (0 < N < 500000, 0 < M < 500000, 1 <= S <= N, 1 <= D <= N)**. In the second line and in the third line there are **M** integers each, all integers in these two lines are between **1** and **N**. In the fourth line, there are **N** integers, all integers in this line are between **1** and **500000**.

There is an undirected graph with **N** nodes and **M** edges. The nodes are numbered from **1** to **N**. The **i**'th edge of this graph is between the **i**'th node in the second line and the **i**'th node in the third line, the edges have no weight, there is no self loop or multi edge. The **i**'th value in the fourth line is the weight of the **i**'th node. The cost of a path is the **sum of the weights of the nodes** in that path. You need to find the cost of a path, if there is any, from node **S** to node **D** with the minimum cost. Solve the problem in **O((N+M)*log(N+M))** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.txt**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one integer. If there is no path satisfying the above rules then this integer is **-1**. Otherwise this integer is the cost of the path you are asked to find above.

Sample Input File 1	Sample Output File 1
6 5 1 5 6 2 3 4 6 2 3 4 1 4	-1

1 2 4 8 9 8	
Sample Input File 2	Sample Output File 2
12 12 2 11 1 1 1 2 2 2 3 3 7 4 11 12 2 3 4 5 6 7 8 9 1 11 5 10 1 2 4 8 1 2 4 8 1 2 8 16	11

#10

Problem Proposed/Prepared by: KKP

You are given a file "**inp5.txt**". The file contains the following info: In the first line there are five integers **N, M, S, D, K (0 < N < 500000, 0 < M < 500000, 1 <= S <= N, 1 <= D <= N, 1 <= K <= N)**. In the second line and in the third line there are **M** integers each, all integers in these two lines are between **1** and **N**. In the fourth line, there are **M** integers, all integers in this line are between **1** and **500000**.

There is an undirected weighted graph with **N** nodes and **M** edges. The nodes are numbered from **1** to **N**. The **i**'th edge of this graph is between the **i**'th node in the second line and the **i**'th node in the third line, their weight is the **i**'th value in the fourth line, there is no self loop or multi edge. The total cost of a path is the sum of the weights of the edges in that path. You need to find the minimum cost to reach node **D** from node **S** such that the path **doesn't** contain the node **K**. Solve the problem in **O((N+M)*log(N+M))** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one integer. If there is no path satisfying the above rules then this integer is **-1**. Otherwise This integer is the minimum cost to reach node **D** from node **S** such that the path **doesn't** contain the node **K.**

Sample Input File 1	Sample Output File 1
6 5 2 5 4 6 2 3 4 5 3 3 4 1 4 1 2 4 8 9	-1
Sample Input File 2	Sample Output File 2
8 9 8 1 6 1 1 2 2 2 3 1 8 7 3 4 5 6 7 8 6 2 4 8 5 9 1 8 9 9 2 1	16

#11

Problem Proposed/Prepared by: KKP, HFN

You are given a file "**inp5.txt**". The file contains the following info: In the first line there are five integers **N, M, S, D, K (0 < N < 500000, 0 < M < 500000, 1 <= S <= N, 1 <= D <= N, 1 <= K <= N)**. In the second line and in the third line there are **M** integers each, all integers in these two lines are between **1** and **N**. In the fourth line, there are **M** integers, all integers in this line are between **1** and **500000**.

There is an undirected weighted graph with **N** nodes and **M** edges. The nodes are numbered from **1** to **N**. The **i**'th edge of this graph is between the **i**'th node in the second line and the **i**'th node in the third line, their weight is the **i**'th value in the fourth line, there is no self loop or multi edge. The total cost of a path is the sum of the weights of the edges in that path. You need to find the minimum cost to reach node **D** from node **S** such that the path contains the node **K**. The path may contain some edges multiple times. Solve the problem in **O((N+M)*log(N+M))** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one integer. If there is no path satisfying the above rules then this integer is **-1**. Otherwise this integer is the minimum cost to reach node **D** from node **S** such that the path contains the node **K**.

Sample Input File 1	Sample Output File 1
7 10 3 7 5 2 2 4 4 1 4 5 4 5 6 3 4 3 1 7 5 7 7 6 7 1 1 6 2 2 3 8 1 4 2	9
Sample Input File 2	Sample Output File 2
5 4 1 4 2 1 1 5 2 5 4 4 3 6 7 2 4	-1

[Solve the same problem for directed graph]

Previous Quiz Questions on Greedy

#1

Problem Proposed/Prepared by: ARD

There are **M** workers, each can work for at most **T** units of time. There are **N** jobs, the **i**'th job starts at **A[i-1]** time and ends at **A[i]** time. One job can not be done by more than one worker. One worker can take more than one job, but the jobs need to be continuous. Can all these workers together finish all these jobs?

Input

The first line contains three space separated integers **M**, **T**, **N** where **0 < M < 10^5**, **0 < T < 10^9**, **0 < N < 10^5**. The second line contains **N+1** space separated integers, the values of **A[i]** where **0 = A[0] <= A[1] <= A[2] <= … <= A[n-1] <= A[n]**.

Output:
"YES" if the workers together can finish all the jobs, "NO" otherwise.

Sample Input/Output:

Sample Input 1	Sample Output 1
2 15 3 0 4 9 19	YES
Sample Input 2	Sample Output 2
2 10 3 0 4 14 19	NO
Sample Input 3	Sample Output 3
2 9 3 0 4 14 19	NO
Sample Input 4	Sample Output 4
2 10 5 0 4 8 12 16 20	NO
Sample Input 5	Sample Output 5
2 12 5 0 4 8 12 16 20	YES

Sample 1 Explanation:
M = 2, T = 15, N = 3, A = [0, 4, 9, 19], output = "YES".
The i'th job takes A[i]-A[i-1] units of time to finish. So the first, second and third jobs take 4, 5 and 10 units of time to finish, respectively.
The first worker works from 9 to 19 in time, the second worker works from 0 to 9 in time. This way, the first worker can do the third job and the second worker can do the first and the second job, no worker exceeds his own time limit. There are other ways to do all the jobs as well.

Sample 2 Explanation:
M = 2, T = 10, N = 3, A = [0, 4, 14, 19], output = "NO".
The first, second and third jobs take 4, 10 and 5 units of time to finish, respectively. There is no way to do all the jobs.
If the first worker works from 0 to 9 in time and the second worker works from 9 to 19 in time, then, the second job is done by both workers which is not allowed.
If the first worker does the first and the third job then his jobs are not continuous which is also not allowed.

#2

Problem Proposed/Prepared by: SAK

You will be given a sequence of jobs where each job has a deadline and a profit. If the job is completed by the deadline, then you will receive the corresponding profit. Every job takes exactly one unit time to complete. Your task is to select the jobs in such a way that the total profit is maximized.

i) Solve it in O(N*D)

ii) Solve it in O((N+D) log N) where (1 <= N <= 10^5) and (1 <= D <= 10^5)

Input
The first line contains an integer **N (1 <= N <= 100)** which is the number of jobs given. Each of the following **N** lines will contain three single space-separated integers **J, D, P. J** denotes the job number, **D** denotes the deadline of the job, and **P** denotes the profit of the job. Here, **1 <= J <= 100, 1 <= D <= 50, 1 <= P <= 10^9.**

Output:
The first line shows the numbers of the jobs selected and completed by their corresponding deadline to maximize profits. The second line shows the total profit based on the jobs selected.

Sample Input/Output

ple Input 1	**Sample Output 1**
4 20 10 40 30	3 1 60
ple Input 2	**Sample Output 2**
5 100 19 27 25 15	3 1 5 142
ple Input 3	**Sample Output 3**
6 40 10 80 20 30 50	1 3 6 170
ple Input 4	**Sample Output 4**
6 60 50 100	4 6 2 3 310

80 70 80	

#3

Problem Proposed/Prepared by: MUNR

You are given a file "**inp3.txt**". The file contains the following info: In the first line there is an integer **N (0 < N < 100000)**. In the second line as well as the third line there are **N** space separated positive integers (each integer is less than 100000).

Suppose there are **N** tasks, the **N** integers in the second line are their starting times, and the **N** integers in the third line are their ending times, respectively. The ending time is greater than the starting time for each task. Two tasks can not be done by the same person if there is some overlap between the two tasks in time. Find the minimum number of persons required to do all the tasks. Do it in **O(N*log(N))** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.out**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one integer (the answer you were asked to find above) in the first and only line.

Sample Input File 1	Sample Output File 1
5 1 2 3 4 5 3 4 5 6 7	2
Sample Input File 2	**Sample Output File 2**
7 1 3 6 10 15 21 28 9 8 9 15 21 28 30	3

Note: In the second sample, one way to do all tasks using 3 persons is the following. First person does (1, 9), (10, 15), (15, 21), second person does (3, 8), (21, 28), third person does (6, 9), (28, 30). There are other ways to do all the tasks using 3 persons, but no way to do all the tasks using 2 persons, so the answer for this sample is 3.

#4

Problem Proposed/Prepared by: KKP

You are given a file "**input3.txt**". The file contains the following info: In the first line there is an integer **N (0 < N < 100000)**. In the second line as well as the third line there are **N** space separated positive integers (each integer is less than 100000).

Suppose there are **N** tasks, the **N** integers in the second line are their lengths in time, and the **N** integers in the third line are their deadlines

as points in time, respectively. You do all these tasks in some order, not more than one at a time. You can possibly finish some task even after the deadline is over. However, the later you do some task, the less points you get. If a task is finished at time **x** and the deadline of the task is **d** then the point you get for doing that task is **(d-x)**. Find the maximum total points you can get by doing all the tasks in some order. Do it in **O(N*log(N))** time.

Write a Python (.py) or Java (.java) or C/C++ (.cpp) code which, if put in the same directory as the given file, produces another file "**00000000.txt**" (**replace the 0's with your ID number**) in the same directory after running the code.

In this file there should be only one integer (the answer you were asked to find above) in the first and only line.

Sample Input File 1	Sample Output File 1
4 4 2 3 4 2 5 7 5	-10
Sample Input File 2	**Sample Output File 2**
5 3 4 5 1 2 1 10 100 1000 10000	11076

Note: In the first sample, if you did the tasks in the order (3, 7), (2, 5), (4, 2), (4, 5) without any gap and you started at 0 time, then the ending times would be 3, 5, 9, 13, respectively. Points would be (7 - 3), (5 - 5), (2 - 9), (5 - 13), respectively, and -11 in total, which would not be optimal for this sample.